

Generative AI: Assignment 2

Total Marks: 100 | Due Date: 20th September, 9PM

I. Problem Statement: Build a Smart Mutual Fund Advisor

On top of the Mutual Funds Report, [sourced](#) from Kaggle

Objective

Create a Gradio-based AI assistant that recommends a mutual fund investment portfolio based on the user's risk and return preferences. The app uses Retrieval-Augmented Generation (RAG) on a structured Mutual Fund product dataset, generating explainable, structured results.

User App (Essential Features)

Inputs

- **Investment Needs:** Free-form text input (*e.g.*, “high return portfolio, only from Aditya Birla fund house”)

Output

- **Suggestion:** A natural language explanation of what's recommended and why
 - **Recommended Mutual Funds:** Structured as a list containing:
 - Scheme Name
 - Fund House
-

Developer-Facing Advanced Settings

Shown under a collapsible **LLM Settings (Advanced)** section in the UI.

Model & Generation Settings

- **Model Selector:** select 2 more open-source / closed-source models
- GPT OSS 120B (openai/gpt-oss-120b)
- **Temperature Slider:** Adjustable between 0 and 2

These settings allow developers to tune how deterministic or creative the model's responses are.

Backend Setup

Vector Store and RAG

- **Embedding Model:** GPT OSS 120B (120B) via Ollama, or other model of your choice
 - **Vector Database:** ChromaDB, or other vector DB of your choice
 - **RAG Workflow:**
 1. Load product catalog using `CSVLoader`
 2. Parse and clean fields into `Document` objects with metadata
 3. Generate embeddings and index documents
 4. Perform similarity search based on user preferences
 5. Use an LLM to generate structured output parsed via `PydanticOutputParser`
-

II. Marking Scheme

Part 1: Implement the Solution (60 marks)

Set environment variables (2.5 marks)

TODO: Load environment variables and API Keys here

Define Chat Model (2.5 marks)

TODO: Model name map for UI Mapping

- ## TODO: Add 2 more models here
- ## TODO: Add 2 more models here

TODO: Model Selector

Define Embedding Model (2.5 marks)

TODO: Create embeddings

Define Vector Store (2.5 marks)

TODO: Initialize ChromaDB with a persist directory set

Load and Preprocess Data (5 marks)

TODO: Keep Scheme Name, Fund House, Scheme Type, Scheme Category, Net Asset Value and Date in content

TODO: Keep Scheme Type, Scheme Category in metadata

Store in Vector DB (5 marks)

TODO: Add UUIDs

TODO: Add documents

Initialize Retriever (5 marks)

TODO: Initialize Retriever below

Define Pydantic Schema (10 marks)

TODO: Define Pydantic schema with type and description

Define Prompt (5 marks)

TODO: Use ChatPromptTemplate.from_template with fields: preferences, context, format_instructions

Test RAG Pipeline Manually (10 marks)

TODO: Retrieve relevant funds

TODO: Joining the funds

TODO: Get LLM

TODO: Create a Runnable Sequence with 3 Runnables - template, LLM and Output Parser (2.5 marks)

TODO: Create dictionary with preferences, context and format_instructions

TODO: Invoke chain and the output of this command should be a dictionary (2.5 marks)

Define RAG Pipeline Function (10 marks)

TODO: Retrieving relevant funds

TODO: Joining the funds

TODO: Get the LLM

TODO: Create a Runnable Sequence with 2 Runnables - template and LLM (2.5 marks)

TODO: Invoke the Runnable Sequence with the preferences, context and format instructions (2.5 marks)

Part 2: Implement the RAG Pipeline in LangGraph (40 marks)

Define State Class (7.5 marks)

TODO: Define State class with preferences, context and answer keys

Define Retrieve Node (7.5 marks)

TODO: Define retrieve node

- ## TODO: Retrieving relevant mutual funds
- ## TODO: Return the relevant parts of the state: context only

Define Generate Node (10 marks)

TODO: Define generate node

- ## TODO: Joining the mutual funds
- ## TODO: Create a Runnable Sequence with 2 Runnables - template and llm_graph (different variable that's defined later)
- ## TODO: Invoke the Runnable Sequence with the preferences, context and format instructions
- ## TODO: Return the relevant parts of the state: answer only

Build and Compile Graph (10 marks)

TODO: Build the graph

- ## TODO: To add
- ## TODO: To add
- ## TODO: To add
- ## TODO: To add
- ## TODO: To add

Test Manually (5 marks – working version; else 2.5 marks)

TODO: Get LLM

TODO: Invoke the graph

III. Submission Guidelines

- Make sure you're not changing the starter code, and just filling in the # TODO statements in the Starter Code.
 - The syntax for the code should align with what has been discussed in class. Very different syntax would not be deemed acceptable (against use of vibe coding).
-
-